

Introduction to Object-Oriented Software Construction

Experiment Guidebook

Experiment 1: Aircraft War System Analysis and Singleton Pattern

Harbin Institute of Technology (Shenzhen)

2025

1. Objective

- (1) Understand the core concepts of object-oriented analysis and design.
- (2) Learn object-oriented analysis and design methods through practical examples.
- (3) Master the skills of drawing UML class diagrams.
- (4) Understand the motivation, structure, and implementation of the Singleton Pattern.

2. Environment

- Operating System: Windows 10
- IDE: IntelliJ IDEA 2025.2
- JDK: OpenJDK 24 or higher
- Tools: PlantUML plugin for IntelliJ IDEA (for drawing UML diagrams)

3. Experiment Tasks

This experiment consists of two main parts:

- (1) **System Analysis:** Analyze the "Aircraft War" game system based on functional requirements and design UML class diagrams.
- (2) **Design Pattern Application:** Identify scenarios suitable for the **Singleton Pattern** and implement it for the Hero Aircraft.

Specific tasks include:

- ✓ Extend the UML class diagram (in uml/Inheritance.puml) by adding core role classes: Hero Aircraft, Enemy Aircrafts, Bullets, and Power-up Supplies; understand inheritance and abstraction in object-oriented design.
- ✓ Add the EliteEnemy class and Power-up Supply classes (e.g., Firepower Supply, Health Supply) to the codebase. Ensure each class includes appropriate attributes and methods necessary for its behavior and integration into the game system.
- ✓ Use PlantUML to draw the Singleton Pattern class diagram for the Hero Aircraft, and save it in the uml directory.
- ✓ Refactor the code with the Singleton Pattern to manage the Hero Aircraft instance (ensuring only one Hero Aircraft exists per game round).

4. Experiment Steps

4.1 System Requirements

1. Character Definitions

Name: Hero Aircraft		Expected Design Pattern
Number of Instances	1	Singleton Pattern
Type	1 type: Hero Aircraft	
Movement Method	Follows the player's mouse movement	
Survival	Survives via HP (health points). Loses partial HP when hit by enemy bullets; loses all HP upon collision with an enemy aircraft.	
Firepower	Automatically fires bullets. Trajectory changes if enhanced by a firepower supply item.	Strategy Pattern
Elimination Condition	Game over when HP reaches 0.	

Name: Enemy Aircraft		Expected Design Pattern
Number of Instances	Unlimited	Factory Pattern
Type	3 types: Mob Enemy, Elite Enemy, Boss Enemy	
Movement Method	Mob and Elite Enemies: Appear at random positions on the screen at a fixed frequency and move downward. Elite Enemies can also move left and right. Boss Enemy: Floats near the top of the screen and moves horizontally until destroyed.	
Survival	Survives via HP (health points). Loses partial HP when hit by enemy bullets; loses all HP upon collision with an enemy	

	aircraft.	
Firepower	Mob Enemy: Does not fire bullets. Elite Enemy and Boss: Fire bullets along predefined trajectories and frequencies.	Strategy Pattern
Elimination Condition	When destroyed, adds corresponding score to total score. Elite or Boss enemies randomly drop a power-up supply.	

Name: Power-up Supply		Expected Design Pattern
Number of Instances	Unlimited	Factory Pattern
Type	2 types: Firepower Supply, Health Supply	
Appearance Method	Randomly appears with a low probability after an enemy aircraft is destroyed.	
Usage Method	Automatically triggered when the hero aircraft collides with it.	
Movement Method	Moves downward at a constant speed.	
Effect	Firepower Supply: Changes the hero aircraft's bullet trajectory and increases firepower for a duration. After expiration, reverts to the original state. Health Supply: Restores a certain amount of HP to the hero aircraft, but cannot exceed its initial maximum HP.	Strategy Pattern
Elimination Condition	Disappears upon collision with the hero aircraft or when moving off the bottom of the screen.	

Name: Bullet		Expected Design Pattern
Number of Instances	Unlimited	
Type	2 types: Hero Bullet, Enemy Bullet	
Movement Method	The flight speed of the bullet.	
Angle	Initial flight direction of the bullet.	
Movement Method	Moves in a straight line at constant speed in a specific direction.	
Elimination Condition	Automatically disappears when hitting the hero aircraft, enemy aircraft, or flying out of the screen boundary.	

2. Game Rules

- (1) Each time the score exceeds a threshold, a Boss aircraft will be spawned.
- (2) Hero aircraft, Elite aircraft, and Boss aircraft fire bullets with different trajectories.
- (3) When an enemy aircraft or the Hero aircraft is hit by a bullet (the bullet disappears), its health points (HP) will decrease. It will crash when its HP drops to 0.
- (4) The Hero aircraft will crash and the game will end if its HP drops to 0 or it collides with any enemy aircraft.
- (5) After an Elite aircraft or Boss aircraft crashes, it will randomly drop a certain supply with a low probability (or no supply may be dropped).
- (6) When different types of enemy aircraft crash, the Hero aircraft will gain the corresponding score.
- (7) The item will take effect automatically when the Hero aircraft collides with it.
- (8) During the game, the Hero aircraft's score and HP will be recorded and displayed in the upper left corner of the interface.

3. Interface and Sound Effect Requirements

- (1) Game Interface: It should have a fresh style and be simple and easy to operate.
- (2) Sound Effects: They can be turned on or off, including:
 - ✓ Game background music
 - ✓ Boss aircraft appearance sound

- ✓ Sound of enemy aircraft being hit by bullets
- ✓ Sound of two types of items taking effect

If sound effects are turned on, the game background music will play in a loop during the game and stop when the game ends. Corresponding sound effects will play when a bullet hits an enemy aircraft, an item takes effect, or the game ends. The background music for the Boss aircraft will play in a loop when it appears, and stop after it crashes or the game ends.

4.2 Template Code Import

This experiment provides a template codebase named AircraftWar-base.

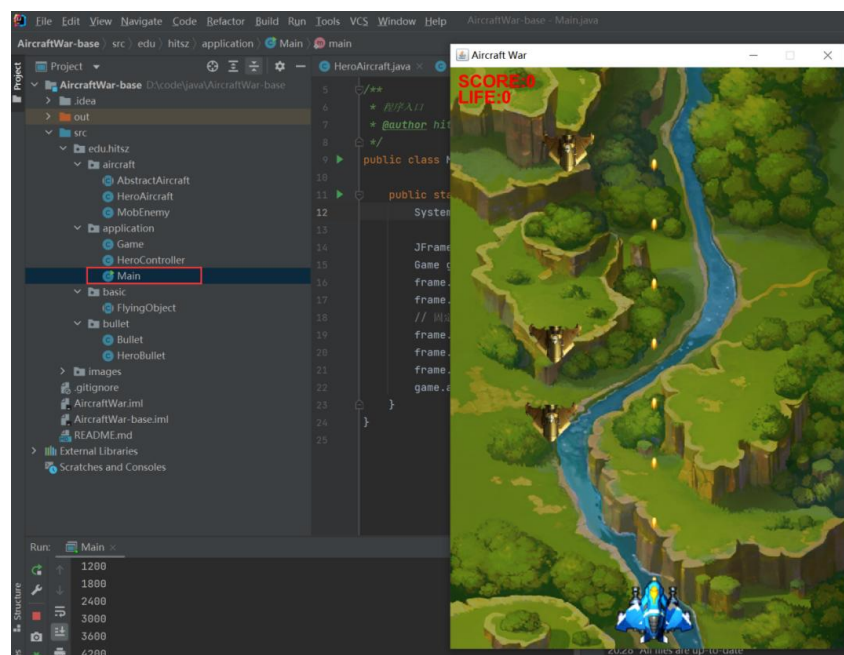
The following core functionalities have already been implemented in the template:

- ✓ Game main interface
- ✓ Movement of the Hero aircraft and Mob Enemy aircraft
- ✓ Bullet firing from the Hero aircraft
- ✓ Collision detection
- ✓ Score tracking and display, along with the Hero's health points (HP)

For instructions on installing the Java integrated development environment IntelliJ IDEA, please refer to file 'Instructions for Experimental Environment Setup' distributed before the experiment.

After installation, you can import and run the AircraftWar-base project. The template code provides a basic, runnable game framework with the Hero aircraft and MobEnemy aircraft already instantiated.

To start the game, run `src/edu/hitsz/application/Main.java`.



4.3 Class Diagram Design

Tasks to be completed:

Based on object-oriented design principles, use the PlantUML plugin to **extend the existing UML class diagram** (uml/Inheritance.puml) by adding the following core game entity classes:

- ✓ Hero Aircraft
- ✓ Enemy Aircraft (including Mob Enemy and Elite Enemy),
- ✓ Bullet (including Hero Bullet and Enemy Bullet),
- ✓ Power-up Supply (including Firepower Supply and Health Supply).

Ensure that the diagram clearly illustrates the inheritance relationships among these classes. The class diagram for Experiment 1 may not include the Boss enemy aircraft.

The initial class diagram showing inheritance relationships is as follows:



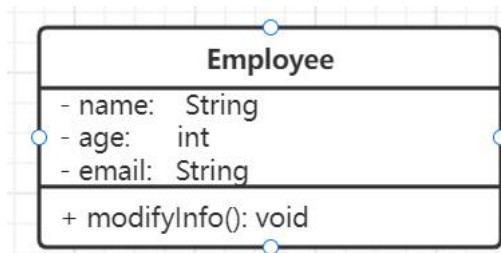
1. Class Diagram

The class diagram is the most commonly used and essential diagram in object-oriented system modeling, and it serves as the foundation for defining other UML diagrams. A class diagram is a static model that primarily depicts the classes and interfaces in a system, along with their static structure and relationships.

In UML, a class is represented as a rectangle divided into compartments by horizontal lines. The compartments contain:

- ✓ the class name,
- ✓ its attributes, and
- ✓ its operations (methods).

For example, an Employee class might include attributes such as name, age, and email, and an operation like modifyInfo().



Below is the corresponding Java implementation:

```
public class Employee {
    private String name;
    private int age;
    private String email;

    public void modifyInfo() {
        .....
    }
}
```

2. Relationships Between Classes

When drawing class diagrams, clarifying the relationships between classes is crucial. Class relationships include **Association**, **Generalization**, **Realization**, and **Dependency**. Among them, Association can be further divided into general Association, Aggregation, and Composition.

(1) Association

It is the most commonly used relationship between classes, representing a structural connection that indicates a link between objects of one class and objects of another class.

① General Association ↓

- ✧ Meaning: A "possession" relationship where one class is aware of another class' s attributes and methods. It can be bidirectional or unidirectional (e.g., teacher and student, customer and product).
- ✧ Code Representation: Member variables.
- ✧ Arrow and Direction: A solid line with a regular arrow, pointing to the "possessed" class. Bidirectional associations may have two arrows or none; unidirectional associations have one arrow.

② Aggregation ↓

- ✧ Meaning: A whole-part relationship where the part can exist independently of the whole (e.g., a car and its tires—tires can exist without the car). Aggregation is a stronger form of association. Grammatically, it is indistinguishable from general association; the distinction lies in logical context.

- ✧ Code Representation: Member variables.
- ✧ Arrow and Direction: A solid line with a hollow diamond, where the diamond points to the "whole."

③ Composition

- ✧ Meaning: A whole-part relationship where the part cannot exist independently of the whole (e.g., a company and its departments—departments cannot exist without the company). Composition is stronger than aggregation; the whole is responsible for the lifecycle of its parts.
- ✧ Code Representation: Member variables.
- ✧ Arrow and Direction: A solid line with a solid diamond, where the diamond points to the "whole."

(2) Generalization

- ✧ Meaning: Also known as inheritance, it describes the relationship between a parent class (base/superclass) and a subclass (derived class). It represents a "general-specific" relationship, where subclasses specialize all attributes and behaviors of the parent class (e.g., a tiger is a type of animal; it has both tiger-specific characteristics and the common traits of animals.).
- ✧ Arrow Direction: A solid line with a triangular arrow, pointing to the parent class.

(3) Realization

- ✧ Meaning: A relationship between a class and an interface, indicating that the class implements all features and behaviors defined by the interface. For example: An interface Vehicle defines an abstract method move(), which is implemented by classes Ship and Car with different details. Ship and Car have a realization relationship with Vehicle.
- ✧ Arrow Direction: A dashed line with a triangular arrow, pointing to the interface.

(4) Dependency

- ✧ Meaning: A "usage" relationship where changes to one thing may affect another that uses it. It applies when one thing relies on another to function. For example: A driver drives a car—the drive() method in the Driver class takes a Car object as a parameter to call car.move(), so Driver depends on Car.

Code Representation: Local variables, method parameters, or calls to static methods.

Arrow and Direction: A dashed line with an arrow, pointing to the "used" class.

Please note the specifications for drawing UML class diagrams, including classes, abstract classes, member variables, methods, visibility, return values, relationship symbols, etc. Draw UML class diagrams strictly according to the requirements of the

guidebook.

Below is an example of how to draw a class diagram using PlantUML.

```
@startuml
'https://plantuml.com/class-diagram

interface Interface
{
}

abstract class AbstractClass
{
}

class Class
{
    + publicField: String
    - privateField: int
    # protectedField: Date
    + {static} staticField: String

    + Constructor(int param1)
    + normalMethod(int param1, String param2): void
    + {abstract} abstractMethod(): int
    + {static} staticMethod(int param1, String param2): String
}

abstract class ParentClassA
class SubClassB
ParentClassA <|-- SubClassB

interface InterfaceC
class ImplementationClassD
InterfaceC <|.. ImplementationClassD

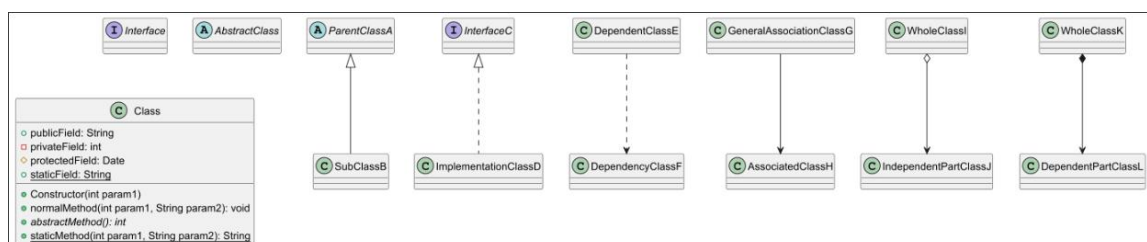
class DependentClassE
class DependencyClassF
DependentClassE ..> DependencyClassF

class GeneralAssociationClassG
class AssociatedClassH
GeneralAssociationClassG --> AssociatedClassH

class WholeClassI
class IndependentPartClassJ
WholeClassI o--> IndependentPartClassJ

class WholeClassK
class DependentPartClassL
WholeClassK *--> DependentPartClassL

@enduml
```



4.4 code refactoring

According to the designed UML class diagram, refactor the code and add the **Elite Enemy** Aircraft class, **two types of Power-up supply** classes in the project. Only the basic requirements for this implementation need to be completed; functional details will be gradually improved in subsequent experiments. The Boss aircraft will not be implemented in this experiment, so it does not need to be included in the UML class diagram or code.

Tasks to be completed:

The version submitted in this experiment must include and correctly implement the following functions:

- a. **Randomly** spawn either a mob or **elite enemy** aircraft at regular intervals.
- b. The elite enemy fires bullets **straight downward** at fixed intervals.
- c. When destroyed, the elite enemy randomly **drops a power-up supply** (or nothing).
- d. When the hero aircraft collides with a power-up, the power-up supply **takes effect automatically**.
- e. The **health supply** restores a certain amount of health to the hero aircraft, but not exceeding its initial maximum.
- f. The **firepower supply** requires no concrete implementation; when activated, it simply prints "FireSupply active!" to the console.

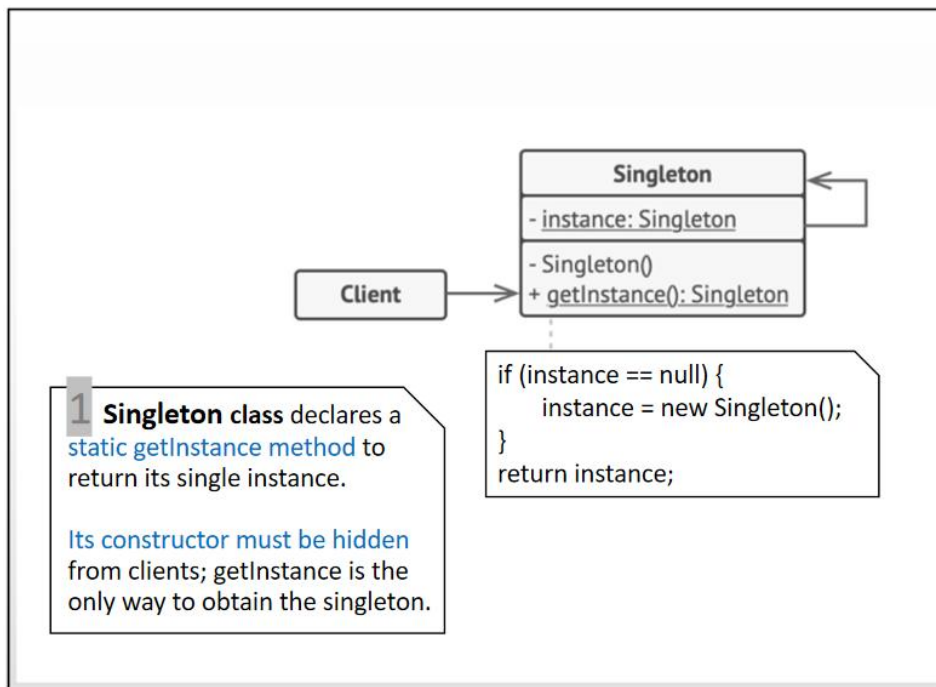
4.5 Singleton Pattern

The Singleton Pattern is a creational design pattern that ensures a class has only one instance and provides a global point of access to that instance.

Tasks to be completed:

(1) Please refer to the following Singleton Pattern structure diagram to **draw the singleton pattern UML class diagram** for the hero aircraft in this game. [The client side does not need to be drawn](#). Save the diagram in the uml directory.

(2) Then Based on your designed UML class diagram, **refactor the code** and create the hero aircraft using the singleton pattern.



The Singleton class declares a static method called `getInstance` to return the same instance of its own class.

The constructor of the singleton must be hidden from client code. Calling the `getInstance` method must be the only way to obtain a singleton object.

Code examples of the Singleton pattern (all three are thread-safe):

(1) Eager initialization

```
public class EagerSingleton {
    private static EagerSingleton instance = new EagerSingleton ();
    private EagerSingleton (){}
    public static EagerSingleton getInstance() {
        return instance;
    }
}
```

(2) Lazy initialization

```
public class LazySingleton {
    private static LazySingleton instance = null;
    private LazySingleton (){}
    public static synchronized LazySingleton getInstance() {
        if (instance == null) {
            instance = new LazySingleton();
        }
        return instance;
    }
}
```

(3) **Double-checked locking (DCL)** **It is recommended to use this.**

```
public class Singleton {
    private volatile static Singleton singleton;
    private Singleton (){}
    public static Singleton getSingleton() {
        if (singleton == null) {
            synchronized (Singleton.class) {
                if (singleton == null) {
                    singleton = new Singleton();
                }
            }
        }
        return singleton;
    }
}
```

5. Assignment Submission

Submission requirements include:

(1) Project Code (the entire project folder "AircraftWar-base")

- UML class diagrams drawn using the PlantUML plugin (saved in the uml directory)
- Fully functional and runnable code

(2) Experiment Report

Please use the provided report template

Place both the project code and the lab report in the same folder, then compress the entire folder into a single archive file (.zip) for submission.

Deadline:

Submit your work to the **HITsz Grader** assignment submission platform **within one week** after the lab session. The exact due date will be specified on the platform.

Login URL: <https://grader.cs-lab.top/>

The default username and password are your student ID.

Note: After uploading, please download your submission to verify that it was uploaded successfully.